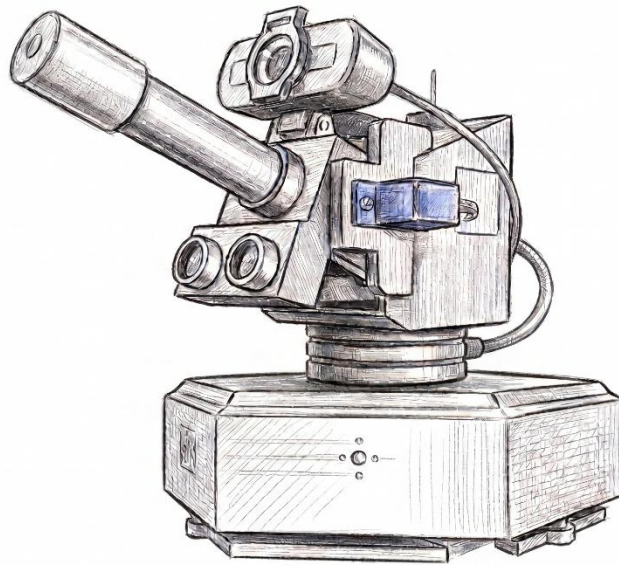


18 DE JUNIO DE 2026



INFORME INTEGRAL DEL PROYECTO

MATERIA ARDUINO: TORRETA DE SEGUIMIENTO DE OBJETOS

Ing. Alfredo G. Rivamar

Ing. Pablo Moliterno

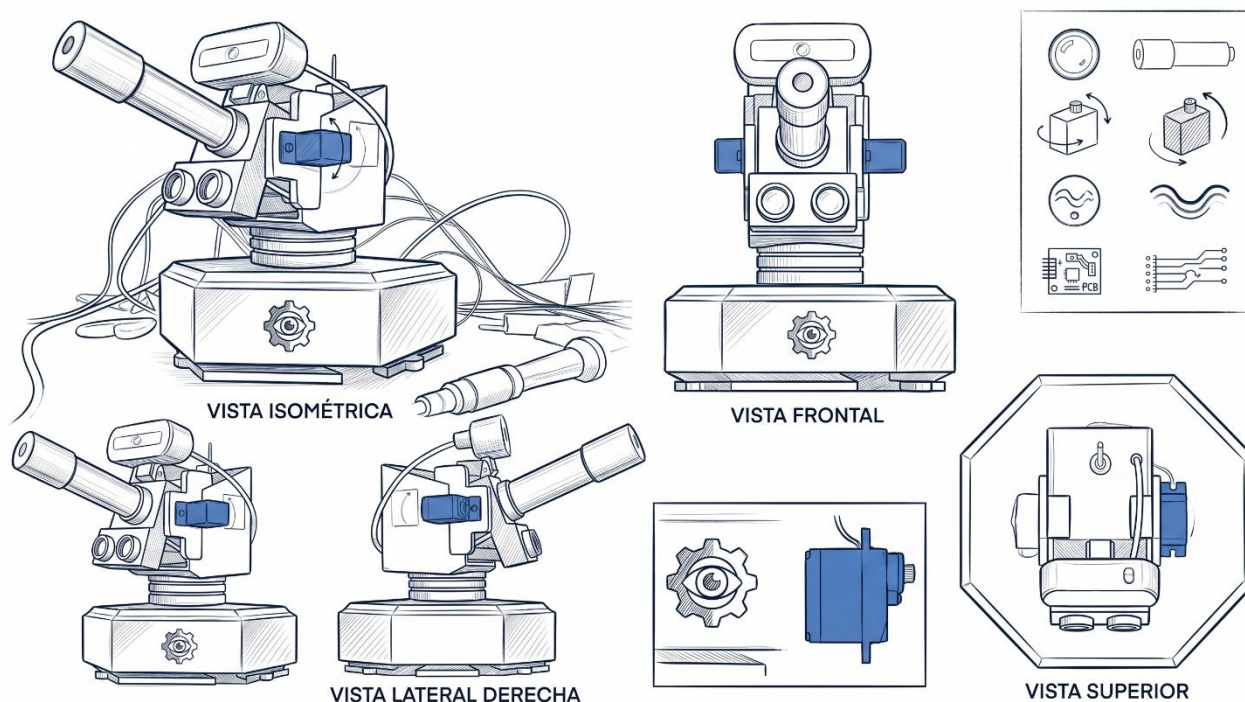
Branko Almeida

Federico Sosa

Agustín Giorlando

Lisandro Toledo

Agustín Lara



Índice

INFORME INTEGRAL DEL PROYECTO	4
Las características técnicas fundamentales de la arquitectura implementada incluyen:	4
OBJETIVOS DEL PROYECTO	4
Objetivo General	4
Objetivos Específicos Cumplidos	4
Casos de Uso y Escenarios de Operación	5
ARQUITECTURA DEL SISTEMA	6
Modelo Distribuido	6
Componentes Principales	6
Justificación Mecánica y Limitaciones de Hardware	7
MÁQUINA DE ESTADOS (FSM)	8
Estados y Transiciones	8
Interrupción por Hardware (ISR)	9
PROTOCOLO DE COMUNICACIÓN (USB-Serie)	10
LAZO DE CONTROL VISUAL (Visual Servoing)	10
VISUAL Y FEEDBACK	11
CARACTERÍSTICAS TÉCNICAS	12
ESTRUCTURA DEL PROYECTO	13
FLUJO DE OPERACIÓN TÍPICO	13
INSTALACIÓN Y USO	14
VALIDACIÓN Y PRUEBAS	15

CONOCIMIENTOS INTEGRADOS.....	18
POSIBLES MEJORAS FUTURAS	18
EQUIPO DE TRABAJO	19
Integrantes del proyecto:	19
NOTAS FINALES	19
Anexo A: Lista de Materiales (BOM):	20
Anexo B: mágenes del modelo en 3D:.....	21
Anexo C: Registro Fotográfico del Ensamblaje	22
Anexo D: lgoritmos Fundamentales del Nodo de Control (Firmware ESP32)	25
Arquitectura de la FSM y Servicio de Interrupción (ISR)	25
Medición de Distancia con Compensación Térmica	25
Lazo de Control de Actuadores (Suavizado)	26
Recepción de Comandos Serie	26

INFORME INTEGRAL DEL PROYECTO

Torreta de Seguimiento y Apuntado con Visión por Computadora

El presente documento detalla el diseño e implementación de una torreta robótica de dos grados de libertad, desarrollada como Trabajo Final Integrador para la asignatura electiva Arduino. El sistema consiste en una **torreta robótica de dos grados de libertad (2-DoF)** capaz de detectar, seguir y "apuntar" a un objetivo móvil mediante visión por computadora, fusionando percepción visual con medición ultrasónica de distancia.

Las características técnicas fundamentales de la arquitectura implementada incluyen:

- Control distribuido: **ESP32** (tiempo real) + **PC con Python/OpenCV** (percepción)
- Comunicación por USB-serie (latencia baja y determinista)
- Dos modos de operación conmutables: **autónomo** (guiado por cámara) y **manual** (joystick)
- Fusión de sensores: ángulos de la cámara + distancia del ultrasonido
- Máquina de estados finitos (FSM) con interrupción por hardware (ISR)
- Compensación térmica en la medición de distancia
- Prototipo de bajo costo con componentes maker

OBJETIVOS DEL PROYECTO

Objetivo General

Diseñar, programar y prototipar una torreta de seguimiento basada en ESP32 que integre múltiples sensores y actuadores en arquitectura distribuida con percepción visual, operación manual/autónoma y respuesta ante eventos críticos por interrupción hardware.

Objetivos Específicos Cumplidos

1. ☒ Arquitectura de hardware justificada y modular
2. ☒ Firmware modular, comentado y organizado como FSM
3. ☒ Al menos una rutina ISR funcional (pulsador del joystick)
4. ☒ Detección y seguimiento en tiempo real mediante OpenCV
5. ☒ Dos grados de libertad: azimut ($\pm 90^\circ$, 180° barrido) y elevación (tilt)

6. ☒ Medición de distancia con compensación térmica (DHT22)
7. ☒ Fusión de sensores (cámara + ultrasonido)
8. ☒ Nivelación y centrado del cañón con MPU6050 (referencia absoluta)
9. ☒ Modo manual conmutable con joystick
10. ☒ Visualización en OLED y aviso sonoro
11. ☒ Validación con prototipo físico

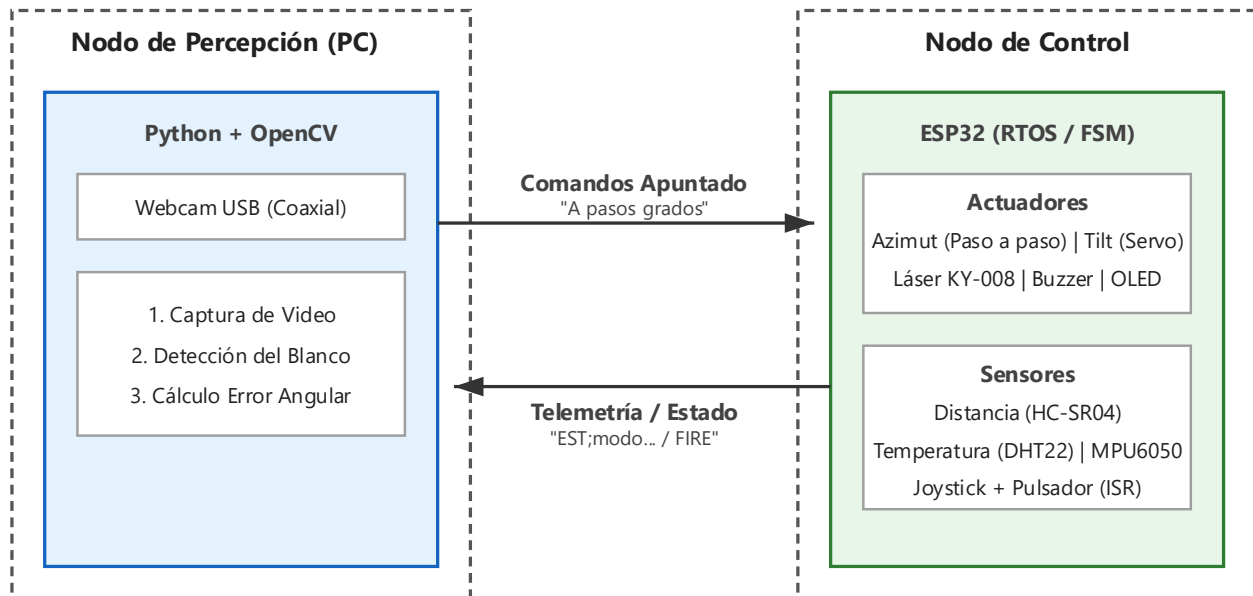
Casos de Uso y Escenarios de Operación

El comportamiento del sistema ante estímulos externos e interacciones del operador se ha modelado mediante los siguientes casos de uso operativos:

- Inicialización (Homing). Al energizar el sistema, el ESP32 inicializa los periféricos y comanda el servomotor hasta que el sensor MPU6050 registra una inclinación nula ($\text{pitch} = 0^\circ$), garantizando la horizontalidad del cañón. Simultáneamente, se establece el origen (cero) del azimut.
- Seguimiento y Disparo Autónomo. Al detectar un objetivo en el campo visual, la PC calcula el error espacial y transmite las correcciones. El sistema converge sobre el blanco y evalúa la distancia. Si se cumple el criterio de rango paramétrico, el hardware acciona el impacto (láser y aviso sonoro).
- Objetivo Centrado Fuera de Rango. Si el blanco está centrado visualmente pero la lectura del HC-SR04 excede el rango de seguridad, el sistema mantiene el seguimiento e inhibe el disparo, evitando falsos positivos.
- Pérdida del Objetivo. Si el blanco abandona el campo visual, el algoritmo suspende las correcciones espaciales. La torreta conserva su última posición conocida a la espera de la readquisición visual.
- Conmutación de Modo. Mediante la detección de una pulsación sostenida en el joystick (gestión por ISR), el operador alterna entre la delegación del control a la PC (Autónomo) y la toma de control directo (Manual).
- Disparo Manual. En modo Manual, una pulsación corta del joystick ejecuta el accionamiento del láser y el buzzer, permitiendo al operador intervenir directamente.
- Saturación por Límite de Sector. Si el seguimiento exige superar el umbral de $\pm 90^\circ$ de azimut, el firmware satura el movimiento para proteger la integridad del montaje físico y el cableado.

ARQUITECTURA DEL SISTEMA

Modelo Distribuido



Componentes Principales

Componente	Función	Justificación
ESP32	Microcontrolador principal, control en tiempo real	16 GPIO, PWM, I2C, SPI; baja latencia para lazo cerrado
Webcam USB	Percepción visual montada coaxialmente con el cañón	Visión directa: centrar blanco en imagen = apuntar sin transformación de coordenadas
HC-SR04	Medición de distancia al objetivo	Define criterio de "objetivo en rango" para disparar; complementa ángulos de cámara
DHT22	Temperatura y humedad	"Para evitar el error sistemático derivado de las variaciones térmicas, el firmware recalcula dinámicamente la velocidad del sonido. Esta corrección se obtiene tomando el valor base de 331.4 metros por segundo, a la cual se le suma el resultado de multiplicar la constante de 0.606 por la temperatura ambiente actual en grados Celsius, leída a través del sensor DHT22."
MPU6050	Acelerómetro + giroscopo 6-ejes	Referencia absoluta de elevación (pitch) para nivelar el cañón en homing
Joystick analógico	Control manual + fuente de ISR	Dos ejes analógicos + pulsador digital para gestos (corto/largo)
SG90	Servo para elevación (tilt)	Lazo abierto; su cero se calibra con el MPU6050

28BYJ-48 + ULN2003	Motor paso a paso para azi-mut	Lazo abierto preciso; se conoce posición contando pasos
Módulo Láser KY-008	Indicador de apuntado / "disparo"	Ligado por transistor NPN 2N2222
Buzzer	Aviso sonoro del impacto	Confirma visualmente que se dispara
OLED SSD1306	Display gráfico 128×64 I2C	Muestra modo, distancia, temperatura, estado

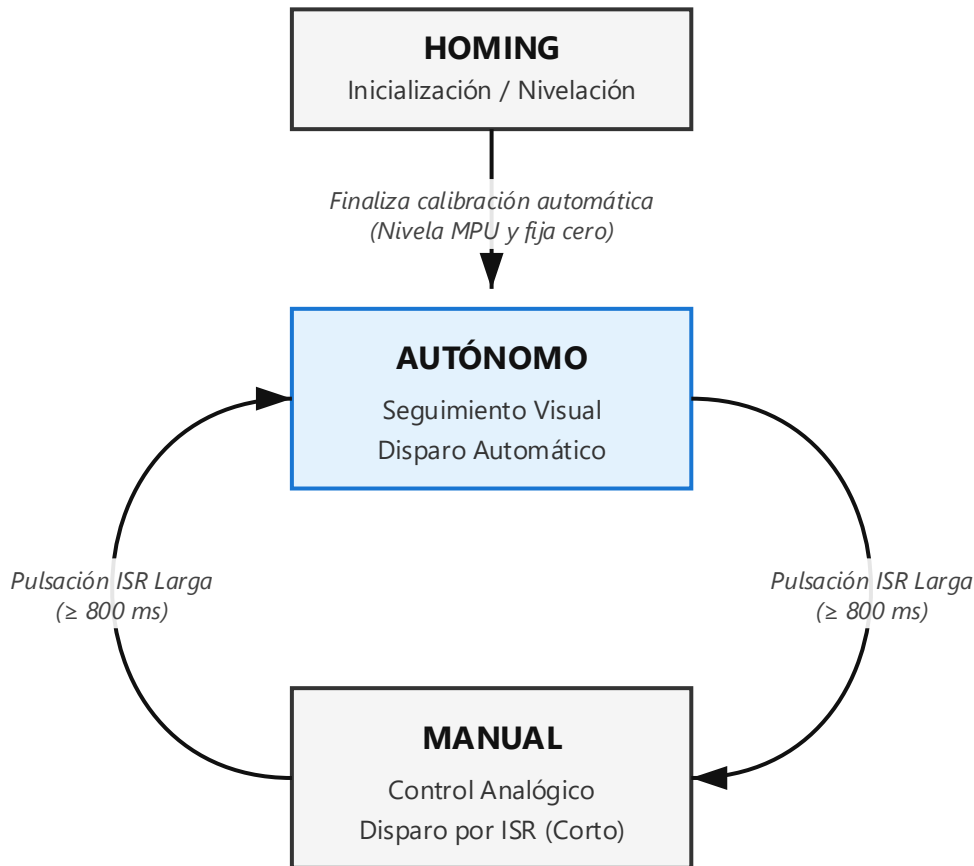
Justificación Mecánica y Limitaciones de Hardware

Sobre la restricción de Azimut (Protección mecánica): Cabe destacar que el eje de azimut ha sido restringido por firmware a un sector útil de $\pm 90^\circ$ (180° de barrido total). Esta limitación mecánica deliberada previene el "efecto guillotina" sobre los cables de comunicación y alimentación (incluyendo el cable USB de la webcam coaxial), evitando torsiones críticas y eliminando la necesidad de implementar complejos anillos rozantes (*slip rings*).

Sobre la Referencia Inercial (MPU6050): Es importante señalar que el sensor MPU6050 carece de magnetómetro integrado, por lo cual no provee orientación absoluta respecto al norte magnético. Su función se restringe exclusivamente a aportar el vector de gravedad para calcular el ángulo de inclinación (*pitch*) y lograr el cero absoluto del servomotor de elevación durante la rutina de *Homing*.

MÁQUINA DE ESTADOS (FSM)

La lógica de control del microcontrolador se rige mediante una Máquina de Estados Finitos (FSM) compuesta por tres estados operativos principales (ver Figura):



Estados y Transiciones

1. HOMING (Inicialización)

- Nivela el cañón moviendo el servo hasta que el MPU6050 lee pitch = 0°
- Fija el cero de azimuth contando pasos
- Inicializa sensores, OLED y enlace serie con PC
- Transición automática a AUTÓNOMO cuando finaliza

2. AUTÓNOMO (Por defecto)

- PC envía correcciones de apuntado basadas en detección visual
- ESP32 mueve 28BYJ-48 (azimut) y SG90 (elevación) para centrar el blanco
- Mide distancia con HC-SR04 y reporta a PC

- Dispara (láser + buzzer) cuando blanco está centrado Y en rango
- Transición a MANUAL por pulsación larga del botón

3. MANUAL (Control por joystick)

- Joystick comanda directamente el movimiento de azimuth y elevación
- Pulsación corta del botón dispara (láser + buzzer)
- Cámara sigue operativa pero sin comandar
- Transición a AUTÓNOMO por pulsación larga del botón

Interrupción por Hardware (ISR)

El **pulsador del joystick** genera interrupciones atendidas por `isrPulsador()`:

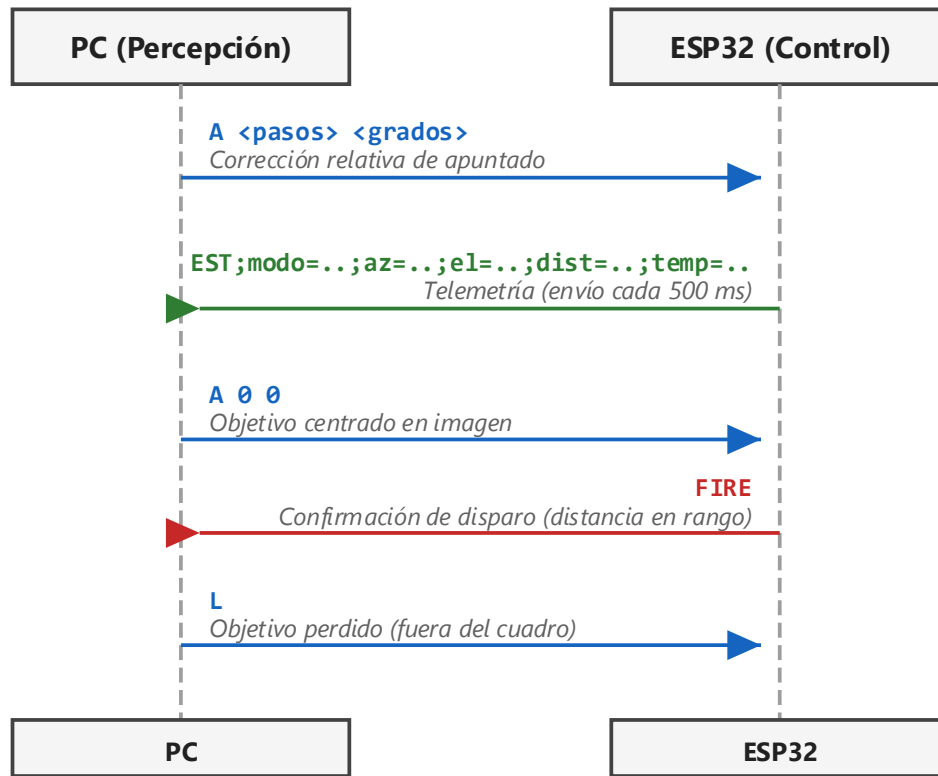
```
void IRAM_ATTR isrPulsador() {
  if (digitalRead(PIN_SW) == LOW) {
    isr_t_bajada = millis(); // registro de tiempo de bajada
  } else {
    unsigned long dur = millis() - isr_t_bajada;
    if (dur >= T_PULSACION_LARGA_MS)      ev_pulsacion_larga = true;
    else if (dur > 30)                    ev_pulsacion_corta = true;
  }
}
```

Se distinguen dos gestos por duración:

- **Pulsación larga** (≥ 800 ms): conmuta MANUAL $\rightleftharpoons$ AUTÓNOMO
- **Pulsación corta** (> 30 ms, < 800 ms): en MANUAL ejecuta disparo; en AUTÓNOMO es automático en rango

PROTOCOLO DE COMUNICACIÓN (USB-Serie)

La PC y el ESP32 se comunican por **USB-série a 115200 baudios** mediante comandos de texto.



Lógica de disparo:

- PC envía A 0 0 (centrado) + ESP32 mide distancia en rango → disparo automático
- En modo MANUAL: pulsación corta dispara directamente
- Disparo dura ~5 s; cooldown de ~7 s evita disparos cascada

LAZO DE CONTROL VISUAL (Visual Servoing)

Es un **lazo proporcional** que anula el error en la imagen:

1. **Captura:** OpenCV detecta el objetivo (por color HSV o detección de mano)
2. **Error:** Calcula desviación en píxeles desde el centro de la imagen
 - Δx horizontal → corrección de azimuth
 - Δy vertical → corrección de elevación
3. **Ganancia:**
 - $PASOS_POR_PX = 0.35$ pasos por píxel de error azimuth
 - $GRADOS_POR_PX = 0.035$ grados por píxel error elevación

4. **Envío:** PC transmite correcciones ~15 veces por segundo
5. **Movimiento:** ESP32 ajusta actuadores; torreta converge sobre el blanco
6. **Repetición:** El ciclo persigue objetivos móviles

La disposición mecánica coaxial de la cámara presenta una ventaja geométrica significativa para el lazo de control: elimina la necesidad de procesar transformaciones de coordenadas espaciales, dado que la acción de centrar el objetivo en el plano de la imagen equivale directamente a la alineación física de los actuadores sobre el blanco.

COMPENSACIÓN TÉRMICA DE DISTANCIA

El HC-SR04 mide distancia por **tiempo de vuelo del sonido**. Sin corregir:

$$v_{\text{sonido}} = 343 \text{ m/s (fijo)} \Rightarrow \text{Error} \approx 0.6 \text{ m/s por } ^\circ\text{C}$$

El firmware **recalcula dinámicamente**:

$$v = 331.4 + 0.606 \cdot T_c \text{ velocidad corregida por temperatura}$$

$$d = \frac{t \cdot 10^{-6} \cdot v \cdot 100}{2} \text{ en cm}$$

El DHT22 mide temperatura en tiempo real, eliminando error sistemático que aumentaría con variación térmica.

VISUAL Y FEEDBACK

OLED SSD1306 (128×64, I2C)

Muestra en pantalla gráfica:

- **Modo actual:** HOMING / AUTÓNOMO / MANUAL
- **Distancia en grande:** si el blanco está visible y en rango
- **Temperatura:** desde el DHT22
- **Estado de la cámara:** objetivo visible / perdido
- **Línea de barrido (radar):** animación visual de azimut
- **Indicadores:** conexión serie, FPS

Indicadores Sonoros y Visuales

- **Láser KY-008:** se enciende en disparo (5 s), indicador visual de apuntado
- **Buzzer:** pulsa durante disparo, audible desde distancia
- **LED integrado del ESP32:** opcional para debug

CARACTERÍSTICAS TÉCNICAS

Grados de Libertad

Eje	Rango	Actuador	Características
Azimut	$\pm 90^\circ$ (180° barrido)	28BYJ-48 + ULN2003	Lazo abierto preciso; posición por conteo de pasos; límite protege cableado
Elevación (tilt)	0°–90° (útil)	SG90 servo	Lazo cerrado calibrado; cero absolutos con MPU6050 en homing

Rango de Medición

- **Distancia:** 2–400 cm (límites HC-SR04)
- **Zona de seguridad:** rango de "objetivo en rango" configurable (defecto: 2–200 cm)
- **Campo visual:** resolución de cámara 640×480

Velocidades

- **Azimut rápido:** 500 pasos/s (aproximación lejana)
- **Azimut fino:** 120 pasos/s (apuntado preciso, <80 pasos del objetivo)
- **Elevación:** ~33°/s (movimiento controlado)
- **Ciclo de control visual:** ~15 correcciones/s

Consumo Aproximado

- ESP32: ~500 mA (típico)
- Motor 28BYJ-48: ~100 mA (en movimiento)
- SG90: ~50–100 mA (según carga)
- Sensores, OLED: ~50 mA
- **Total:** ~800 mA–1 A en operación normal (fuente USB 2A recomendada)

ESTRUCTURA DEL PROYECTO

arduino_torreta/	
└─ firmware/	
└─ firmware.ino	# Nodo de Control: FSM, ISR y sensores
└─ pc/	
└─ seguimiento.py	# Nodo de Percepción: OpenCV y serial
└─ requirements.txt	# Dependencias del entorno Python
└─ hsv.json	# Rango de calibración de color
└─ mira.json	# Offset de paralaje cámara-láser
└─ README.md	# Manual de uso de la interfaz de PC
└─ docs/	
└─ DESCRIPCION_PROYECTO.md	# Especificación técnica detallada
└─ MONTAJE_COMPONENTES.md	# Guía de ensamblaje mecánico
└─ diseño_maqueta/	# Modelos 3D (STEP, STL)
└─ wokwi/	# Entorno de simulación del circuito
└─ i2cscan/	# Herramienta de diagnóstico I2C
└─ oledtest/	# Prueba de hardware para pantalla OLED
└─ _build_pdf.py	# Script para compilar documentación
└─ pyproject.toml	# Configuración del proyecto Python
└─ turret.yaml	# Configuración general
└─ README.md	# Abstract y presentación del repositorio

FLUJO DE OPERACIÓN TÍPICO

Escenario: Seguimiento autónomo con disparo

1. Puesta en marcha (HOMING)

- Usuario enciende el sistema
- ESP32 inicializa sensores, enlace serie, OLED
- Servo se mueve hasta que MPU6050 lee pitch = 0° → cañón horizontal
- Se fija el cero de azimut
- Transición automática a AUTÓNOMO
- PC inicia captura y envía la primera telemetría

2. Objetivo entra en cámara (AUTÓNOMO)

- OpenCV detecta el blanco por color o mano
- PC calcula error respecto al centro
- Envía correcciones de azimut y elevación
- ESP32 mueve actuadores; torreta converge

3. Blanco se centra (AUTÓNOMO, continuidad)

- PC envía A 0 0 (centrado)
- ESP32 mide distancia con HC-SR04 (corregida por T del DHT22)
- Si distancia está **en rango** (ej. 50–200 cm):
 - Activa láser y buzzer (~5 s)
 - OLED muestra "Objetivo alcanzado"
 - Envía "FIRE" a PC

4. Objetivo se mueve (AUTÓNOMO)

- Lazo sigue en tiempo real
- Si sale del campo visual: PC envía "L", torreta mantiene posición

5. Conmutación a MANUAL

- Usuario hace pulsación larga (800+ ms) del pulsador
- FSM conmuta a MANUAL
- Joystick toma control directo
- Pulsación corta dispara
- OLED indica modo MANUAL

INSTALACIÓN Y USO

Hardware

1. Ensamblar componentes según MONTAJE_COMPONENTES.md
2. Conexiones según docs/CONEXIONES.md (pines del ESP32)
3. Cargar firmware via Arduino IDE o PlatformIO

Software (PC)

```

# Clonar o descargar el repositorio
cd arduino_torreta/pc

# Crear entorno virtual
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate

# Instalar dependencias
pip install -r requirements.txt

# Ejecutar (puerto automático)
python seguimiento.py

# Opciones de línea de comandos
python seguimiento.py --puerto COM3          # puerto específico
python seguimiento.py --sin-serial             # sin ESP32 (solo visión)
python seguimiento.py --calibrar               # recalibrar color HSV

```

Calibración

1. Iniciar: python seguimiento.py --calibrar
2. Ajustar barras HSV hasta que el blanco sea blanco y el fondo negro en la máscara
3. Guardar con s (se guarda en hsv.json)
4. Tecla r alterna entre detección por color y detección de mano (MediaPipe)

Teclas durante operación

- **q**: salir
- **c**: recalibrar color
- **r**: alternar objetivo color/mano
- **h**: mostrar/ocultar ayuda
- **w/a/s/d**: ajustar offset de mira (paralaje cámara-láser)

VALIDACIÓN Y PRUEBAS

Pruebas completadas

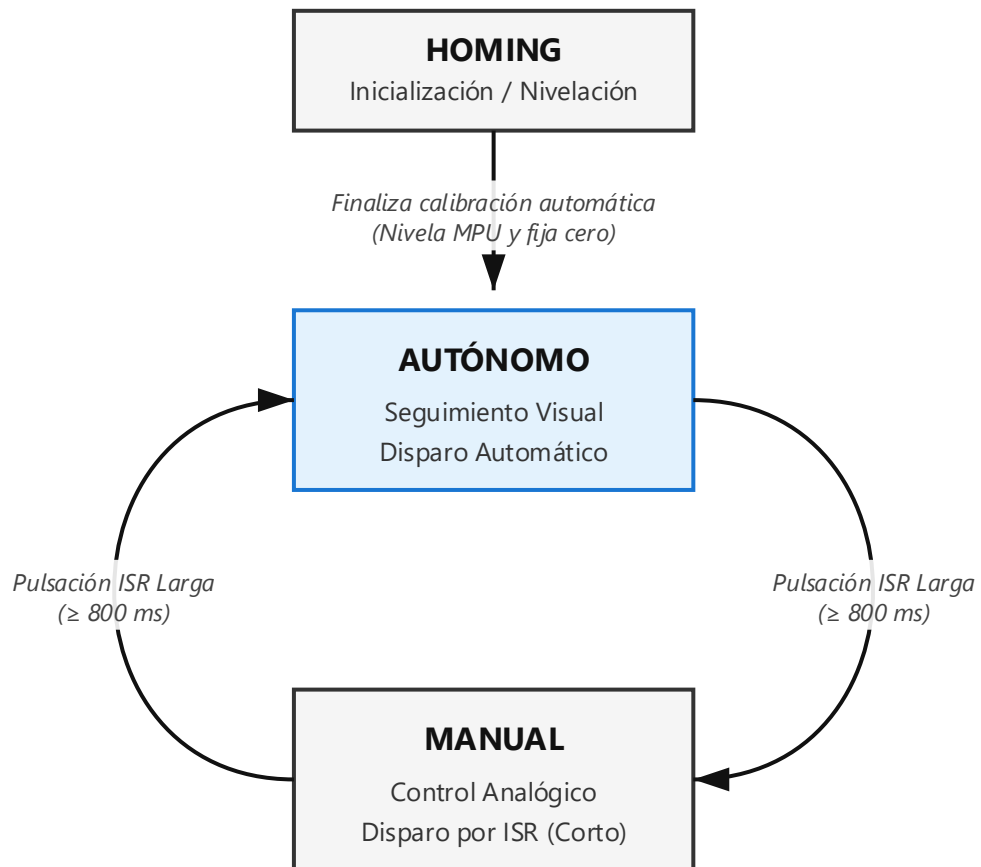
- ✓ Homing: cañón se nivela correctamente a 0° de elevación
- ✓ Azimut: giro en ambas direcciones, límite de $\pm 90^\circ$ respetado
- ✓ Elevación: servo responde a comandos, rango 0–90°

- ✓ HC-SR04: medición de distancia funcional
- ✓ Compensación térmica: validada contra cambios de temperatura
- ✓ ISR: pulsador detecta gestos cortos y largos
- ✓ Comunicación serie: comandos y telemetría fluyen correctamente
- ✓ Lazo visual: detección de color HSV, seguimiento en tiempo real
- ✓ Detección de mano: MediaPipe Hands funcional
- ✓ Disparo: láser y buzzer activan en condiciones (centrado + en rango)
- ✓ Modos: conmutación MANUAL $\rightleftharpoons$ AUTÓNOMO operativa
- ✓ OLED: muestra correctamente estado, distancia, temperatura
- ✓ Simulación Wokwi: compatible con el protocolo

Limitaciones conocidas

- Servo SG90 en lazo abierto; compensado con MPU6050 en homing
- Foco fijo de la cámara: ajustar distancia mínima a ~40 cm

- Barrido limitado a $\pm 90^\circ$ para proteger cableado



CUMPLIMIENTO DE REQUISITOS DE LA CÁTEDRA

Requisito	Cómo se cumple
Firmware modular y comentado	Códigos organizados en funciones por subsistema (sensores, actuadores, comunicación, FSM). Cada función tiene su propósito explícito.
Máquina de estados finitos	Estados HOMING / AUTÓNOMO / MANUAL con transiciones explícitas (FSM).
≥ 1 interrupción de hardware	ISR del pulsador del joystick: detecta gestos cortos y largos para conmutación y disparo.
Justificación de componentes	Tabla detallada en sección 5 de DESCRIPCION_PROYECTO.md; justifica cada sensor/actuador.
Fusión de sensores	Cámara aporta ángulos e identidad; ultrasonido aporta distancia. Combinación crítica para disparo.
Respuesta a evento crítico	Interrupción del pulsador conmuta modos y dispara.

Modo manual	Control por joystick conmutable con pulsación larga; gesto corto dispara en MANUAL.
--------------------	---

CONOCIMIENTOS INTEGRADOS

Este proyecto demuestra competencias en:

- **Electrónica:** diseño de circuitos, interfacing de sensores/actuadores, drivers (ULN2003, NPN)
- **Microcontroladores:** ESP32, periféricos (GPIO, I2C, SPI, PWM, interrupciones)
- **Programación:** C++ (firmware), Python (visión), protocolos de comunicación
- **Percepción visual:** detección de color HSV, seguimiento, OpenCV
- **Machine Learning:** detección de manos (MediaPipe)
- **Teoría de control:** lazo realimentado, proporcionalidad, compensación térmica
- **Comunicación:** USB-serie, protocolos de texto
- **Ingeniería mecánica:** montaje coaxial, transmisiones (motor paso a paso, servo)
- **Trabajo colaborativo:** proyecto de equipo multidisciplinar

POSIBLES MEJORAS FUTURAS

1. **Escaneo 3D:** usar azimuth + elevación + distancia para generar nube de puntos (hardware listo)
2. **Predicción de movimiento:** incluir filtro de Kalman para anticipar la trayectoria
3. **WiFi en lugar de USB:** para mayor movilidad (compromiso: latencia)
4. **Almacenamiento de mapas:** guardar escaneos 3D para análisis posterior
5. **Múltiples objetivos:** priorización y seguimiento simultáneo
6. **Retroalimentación visual:** LED anillo alrededor del láser para feedback
7. **Integración con ROS:** para robots colaborativos

EQUIPO DE TRABAJO

Integrantes del proyecto:

- Branko Almeida
- Federico Sosa
- Agustín Giorlando
- Lisandro Toledo
- Agustín Lara

Institución: Universidad Tecnológica Nacional — Facultad Regional San Rafael **Carrera:** Ingeniería en Sistemas de Información **Asignatura:** Electiva: Arduino **Tipo:** Trabajo Final Integrador (TFI) **Año:** 2026

NOTAS FINALES

El prototipo desarrollado integra de manera funcional conceptos avanzados de diversas disciplinas de la ingeniería (hardware, firmware, visión, control) en un sistema operativo y demostrativo. Su arquitectura distribuida permite experimentar con lazo de realimentación visual, fusión de sensores y máquinas de estado en un contexto tangible.

El código es **modular y extensible**, facilitando futuras mejoras y adaptaciones. La documentación completa (especificación, guía de montaje, protocolo) permite a terceros replicar o derivar el proyecto.

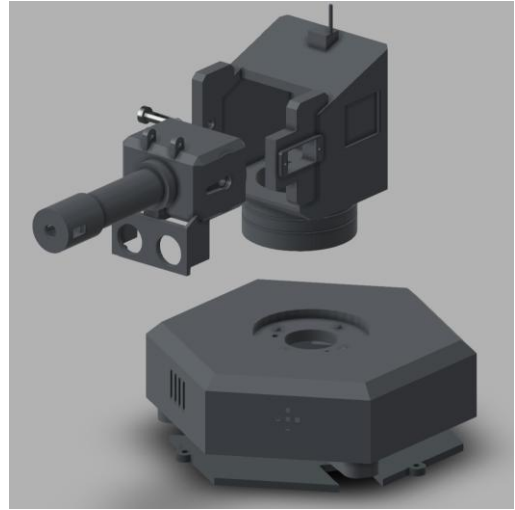
Anexo A: Lista de Materiales (BOM):

Ítem	Componente / Descripción	Cantidad	Función Principal
1	ESP32 (Placa de desarrollo)	1	Microcontrolador central (RTOS).
2	Servomotor SG90	1	Actuador para el eje de elevación (Tilt).
3	Motor paso a paso 28BYJ-48 + Driver ULN2003	1	Actuador para el eje de azimut (Pan).
4	Sensor Ultrasónico HC-SR04	1	Medición de distancia al objetivo.
5	Sensor DHT22	1	Medición de temperatura para compensación.
6	Módulo IMU MPU6050	1	Nivelación absoluta de elevación (Pitch).
7	Joystick Analógico (con pulsador)	1	Interfaz de control manual e interrupciones.
8	Pantalla OLED SSD1306 (0.96", I2C)	1	Display gráfico de telemetría e interfaz.
9	Módulo Láser KY-008	1	Indicador visual de disparo.
10	Transistor NPN 2N2222	1	Llave de conmutación de 5V para el láser.
11	Resistencia 1 kΩ	1	Limitador de corriente para la base del transistor.
12	Zumbador Piezoeléctrico (Buzzer)	1	Indicador acústico de impacto.
13	Webcam USB genérica	1	Captura de video para el nodo de PC.

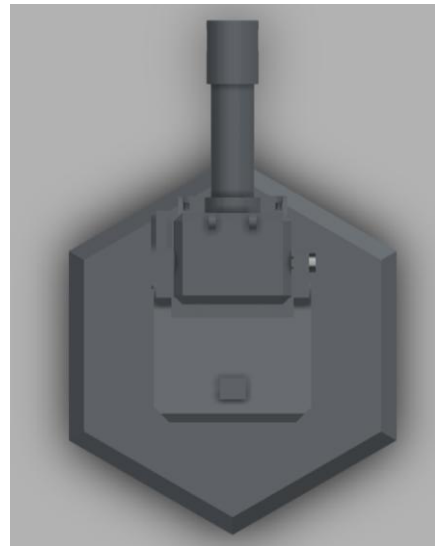
Anexo B: mágenes del modelo en 3D:



Figura 1. Renderizado 3D de la torreta: Vista isométrica general.



Renderizado 3D de la torreta: Despiece de componentes mecánicos.



Figuras 3 y 4: Renderizado 3D de la torreta: Vista frontal y superior

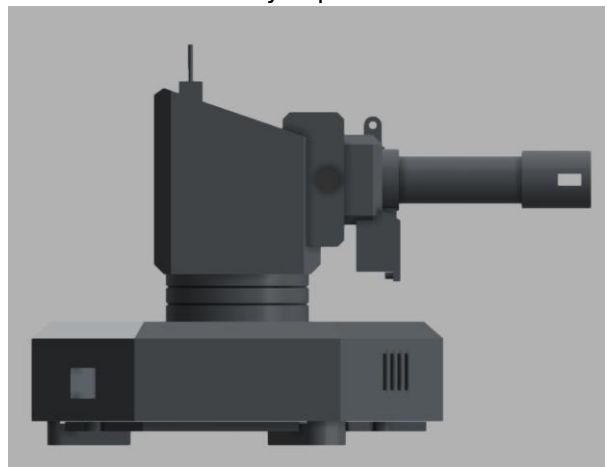
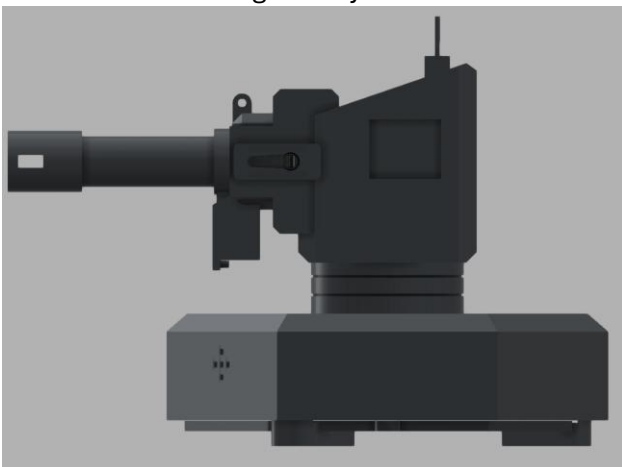
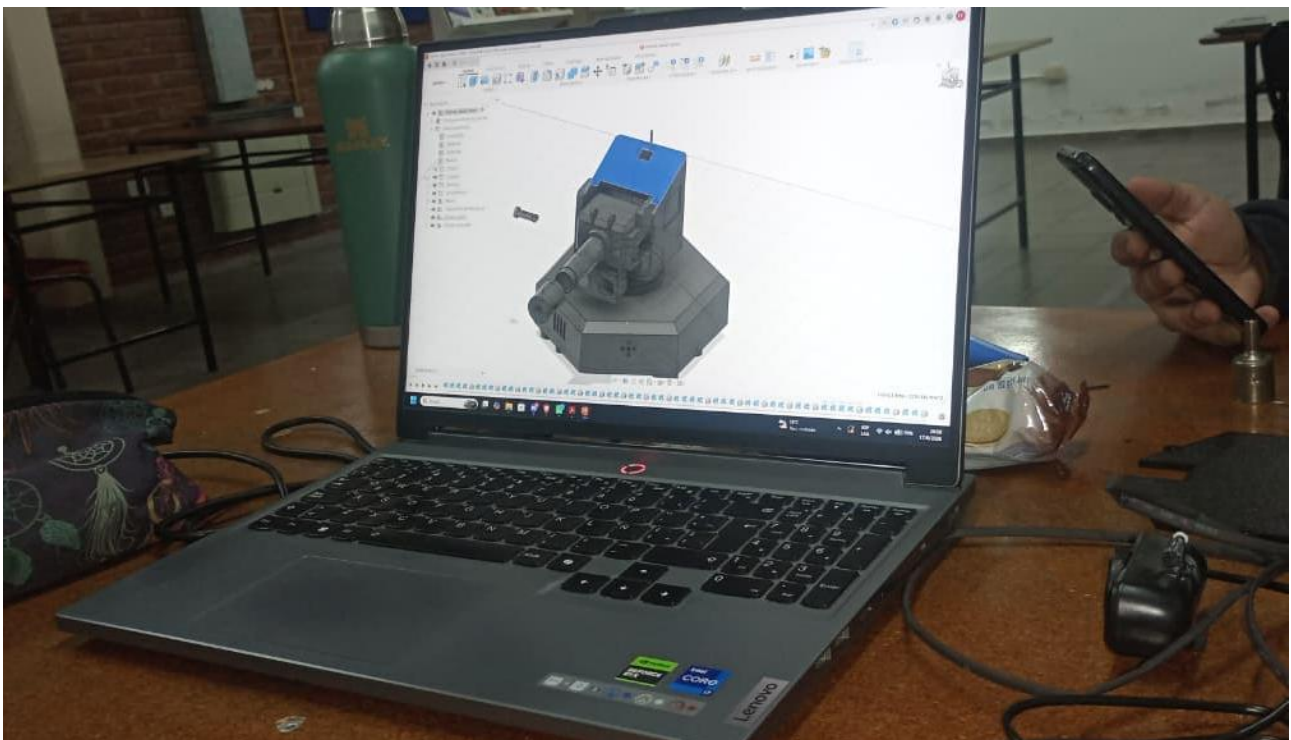


Figura 5 y 6. Renderizado 3D de la torreta: Planta y perfil (vistas ortogonales).

Anexo C: Registro Fotográfico del Ensamblaje

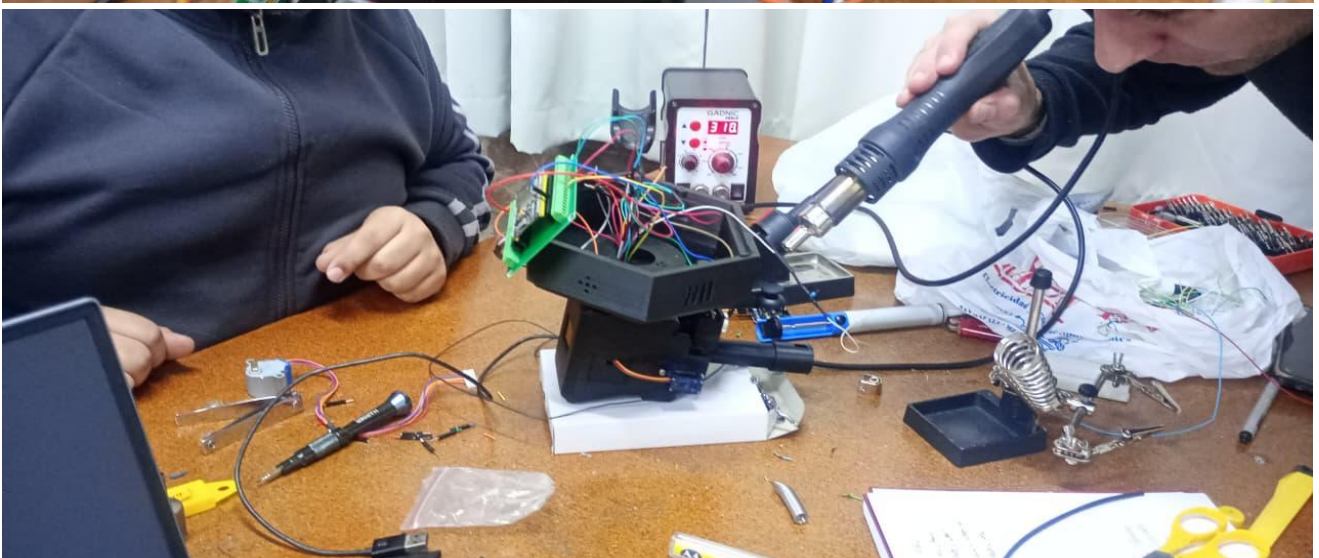
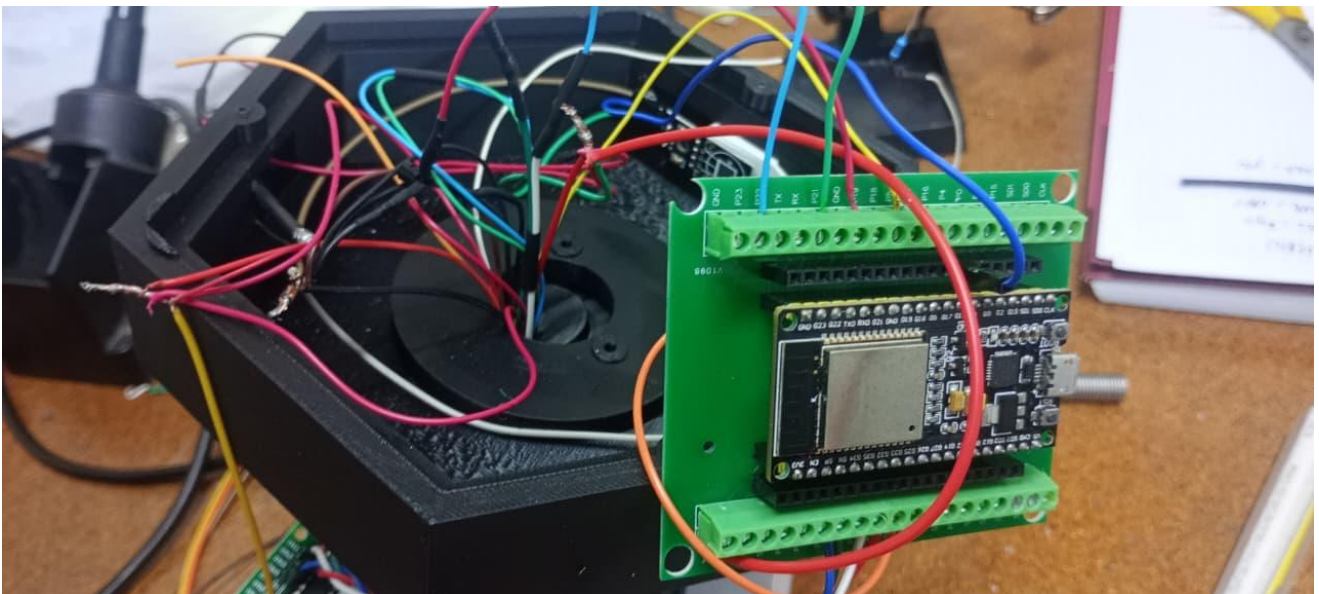
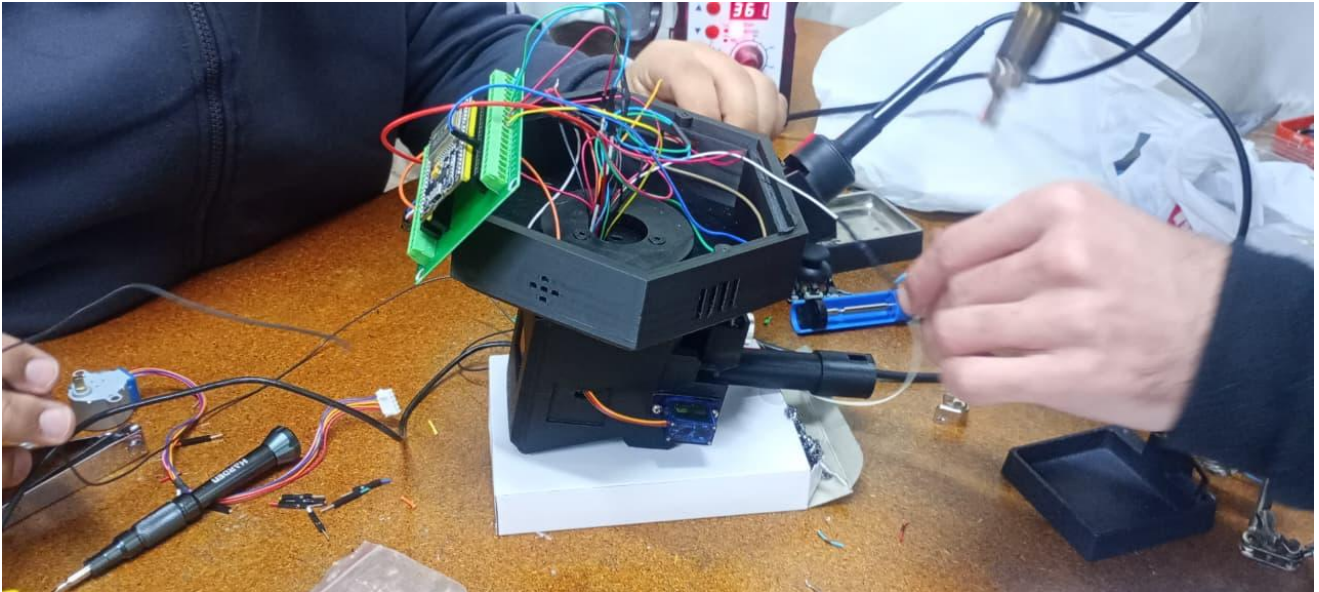


Pruebas de integración iniciales



Diseño asistido por computadora (CAD) y modelado de las piezas estructurales de la maqueta.





Proceso de soldadura, montaje de componentes electrónicos y acople de los actuadores (servo y motor paso a paso)

Anexo D: lgoritmos Fundamentales del Nodo de Control (Firmware ESP32)

Arquitectura de la FSM y Servicio de Interrupción (ISR)

Este bloque muestra cómo se procesan eventos de hardware críticos sin utilizar retardos (`delay()`), discriminando entre una pulsación larga (cambio de modo) y una pulsación corta (disparo manual).

```
// ----- ISR del pulsador (gesto corto/largo) -----
volatile unsigned long isr_t_bajada = 0;
volatile bool ev_pulsacion_corta = false;
volatile bool ev_pulsacion_larga = false;

void IRAM_ATTR isrPulsador() {
  if (digitalRead(PIN_SW) == LOW) {
    isr_t_bajada = millis();
  } else {
    unsigned long dur = millis() - isr_t_bajada;
    if (dur >= T_PULSACION_LARGA_MS)      ev_pulsacion_larga = true;
    else if (dur > 30)                    ev_pulsacion_corta = true; // anti-rebote
  }
}

// ----- Procesamiento de la FSM en el lazo principal -----
void loop() {
  // Evaluación de gestos procesados por la ISR
  if (ev_pulsacion_larga) {
    ev_pulsacion_larga = false;
    estado = (estado == ST_AUTONOMO) ? ST_MANUAL : ST_AUTONOMO;
    blancoVisible = blancoCentrado = false;
  }
  if (ev_pulsacion_corta) {
    ev_pulsacion_corta = false;
    if (estado == ST_MANUAL) disparar();
  }

  // ... (Resto de la lógica del loop)
}
```

Medición de Distancia con Compensación Térmica

Este bloque demuestra la aplicación de conceptos físicos para corregir la velocidad del sonido basándose en la temperatura ambiente.

```
// Distancia con compensación térmica:  $v(T) = 331,4 + 0,606 \cdot T$  [m/s]
float leerDistanciaCm(float tempC) {
  digitalWrite(PIN_TRIG, LOW); delayMicroseconds(2);
  digitalWrite(PIN_TRIG, HIGH); delayMicroseconds(10);
  digitalWrite(PIN_TRIG, LOW);
```

```

long us = pulseIn(PIN_ECHO, HIGH, 30000);
if (us == 0) return -1;

float v = 331.4 + 0.606 * tempC;
return us * 1e-6 * v * 100.0 / 2.0;
}

```

Lazo de Control de Actuadores (Suavizado)

Este método evidencia el uso de temporizadores no bloqueantes para suavizar la cinemática del servomotor y evitar picos de consumo o movimientos bruscos.

```

// Acerca el servo a elevacionObjetivo de a EL_PASO_GRADOS por tick: movimiento
// suave y controlado (sin saltos), lo que amortigua el overshoot del lazo.
void actualizarServo() {
    if (millis() - t_ultimoServo < T_SERVO_MS) return;
    t_ultimoServo = millis();

    if (elevacion < elevacionObjetivo) elevacion = min(elevacion + EL_PASO_GRADOS,
elevacionObjetivo);
    else if (elevacion > elevacionObjetivo) elevacion = max(elevacion - EL_PASO_GRADOS,
elevacionObjetivo);

    servoTilt.write(elevacion);
}

```

Recepción de Comandos Serie

Este fragmento ilustra la interfaz de comunicación bidireccional, decodificando los comandos relativos de apuntado emitidos por el nodo de percepción.

```

// Comandos de la PC:
// A <pasos> <grados> : correccion relativa de apuntado
// L : blanco perdido (conserva la posicion)
void procesarSerial() {
    if (!Serial.available()) return;
    String linea = Serial.readStringUntil('\n');
    linea.trim();
    if (linea.length() == 0) return;

    if (linea.startsWith("A")) {
        long dAz = 0; int dEl = 0;
        sscanf(linea.c_str(), "A %ld %d", &dAz, &dEl);
        if (estado == ST_AUTONOMO) {
            moverAzimut(dAz);
            moverElevacion(dEl);
            blancoVisible = true;
            blancoCentrado = (dAz == 0 && dEl == 0);
        }
    }
}

```



```

} else if (linea.startsWith("L")) {
    blancoVisible = blancoCentrado = false;    // Queda a la espera
}
}

```

